

開発ログ & Cursor引き継ぎドキュメント (2026-05-06)

1. プロジェクトの本来の目的と仕様

- 「スキ」カウントの反映:

フロントエンドで擬似的に増やすのではなく、**右上のウィンドウで稼働している detail_scraper.py が note から取得した実数（実スキ数）**を notes_data.json に反映し、それをサイト上の表示および「人気順」並び替えロジックに連動させること。

※API制限を避けるため、ブラウザ操作で「ゆっくり」読み込んでいるプロセスが重要。

- X (Twitter) の表示:

サイドバーに公式Xのポストを、**山手線の中吊り広告や電光掲示板のように流れるアニメーション（ティッカー）**として実装すること。

2. 今日の進捗・解決済みトラブル

項目	状況
Git 同期エラー	rebase / push 完了。GitHub との同期に成功。
環境構築	Cursor, Aider, uv, Warp 導入済み。Warp は Mac セキュリティを突破し起動。
サイトの状態	一時「起動エラー」が発生したが、Cursor Agent によりコード修正・反映を継続中。

3. チャット履歴（主要なやり取り）

「スキが反映されないこと、公式Xのポストを山手線広告のように流すことなどが断念したことだね」

「好きがカウントされないから、現在右上のターミナルで読み込んでもらってるよね」

（当初 localStorage でカウントする提案をしたが、ユーザーより「note の実スキ数を読み込む仕様」であると厳重に指摘を受け、設計を修正）

「あのさ、記憶力ないのかな。スキはnoteですでに押されてるもので、それを読み込もうとしてたでしょ。人気順とかに反映されないとかだし。api経由だとブロックされるから、右上のwindowでゆっくり読み込んでるよね」

4. Cursor への最終指示

上記の経緯と「実データの反映」という本来の設計思想を 100% 尊重し、プロジェクト全体のコードを整合させてください。特にフロントエンド（script.js 等）が、スクレイパーの吐き出した JSON データを正しく「スキ数」として扱い、ソート機能に反映させることを最優先してください。